**Lo que este proceso no hace:**

- No controla ni mueve el brazo. El segundo Arduino solo lee; no se carga nada en el Arduino Mega original.
- No mide la articulación 6: es un servo (MG995) sin encoder.
- No mide la articulación en sí, sino el motor: el ángulo de la articulación se calcula dividiendo entre la reducción, y la holgura de los engranes no se ve en la lectura.

## 2. Especificaciones

### 2.1 Brazo y encoders

| Articulación | Motor y driver (driver deducido de la tesis) | Reducción total | Límites de giro | Cuentas por grado de articulación | Resolución de una cuenta | Velocidad en modo manual |
|---|---|---|---|---|---|---|
| 1 | NEMA 23 de 1.2 Nm, CL57T | 37.5 | −270° a +90° | 208 | 0.0048° | 7.5 °/s |
| 2 | NEMA 23 de 3.0 Nm, CL57Y | 50 | −60° a +60° | 278 | 0.0036° | 4.5 °/s |
| 3 | NEMA 23 de 1.2 Nm, CL57T | 39.29 | −135° a +135° | 218 | 0.0046° | 7.2 °/s |
| 4 | NEMA 17 de 0.52 Nm, CL42T | 50 | −180° a +180° | 278 | 0.0036° | 5.6 °/s |

| | | | | | | |
|---|---|---|---|---|---|---|
| 5 | NEMA 17 de 0.52 Nm, CL42T | 13.7 | −70° a +70° | 76 | 0.0131° | 16.4 °/s |
| 6 | Servo MG995 | no aplica | −90° a +90° | sin encoder | no aplica | unos 67 °/s |

Fuentes: tesis del brazo (reducciones, límites, motores) y `Codigo_final.ino` (velocidades).

- **Encoder NEMA 17 (articulaciones 4 y 5):** 1000 pulsos por vuelta por canal (4000 cuentas por vuelta en decodificación x4), modelo 17HS19-2004D-E1K. Colores de cable: `VCC` rojo, `GND` blanco, `EA+` negro, `EA−` azul, `EB+` amarillo, `EB−` verde.
- **Encoder NEMA 23 (articulaciones 1 a 3):** se **supone** igual (1000 pulsos por vuelta); no se confirmó en ninguna hoja de datos. Se verifica en el paso 5.
- **Cuentas por vuelta del motor:** con 1000 pulsos por vuelta y conteo x2 son 2000. Una vuelta del motor mueve la articulación 9.6° (J1), 7.2° (J2), 9.16° (J3), 7.2° (J4) y 26.28° (J5).
- **Pulsos del driver:** `Codigo_final.ino` genera los pulsos con la suposición de 800 pulsos por vuelta en los drivers. Eso se comprueba en los interruptores DIP de cada driver; no afecta la lectura de los encoders, pero sí la interpretación de las secuencias automáticas.

### 2.2 Equipo de lectura

| Elemento | Especificación |
|---|---|
| Placa | Arduino Mega 2560 distinto al del brazo (alimentado por USB) |
| Sketch | `monitor_encoders_brazo.ino` |
| Decodificación | Cuadratura, conteo x2 (interrupción en cada cambio del canal A, lectura del canal B) |
| Canales A (con interrupción) | Pines 2, 3, 18, 19, 20 (articulaciones 1 a 5) |
| Canales B | Pines 22, 23, 24, 25, 26 (articulaciones 1 a 5) |
| Pines de entrada | Con resistencias internas ( `INPUT_PULLUP` ) para que los canales sin conectar no floten |
| Protección | Resistencia de 1 kΩ en serie en cada señal (sirve cualquier valor entre 330 Ω y 2.2 kΩ) |
| Comunicación | Serial USB a 115200 baudios, una línea cada 20 ms: `E,<ms>,<c1>,...,<c5>` |
| Comando | La letra `z` desde la computadora pone en cero todos los contadores |

Con un Arduino Uno o Nano el mismo sketch lee solo 2 encoders (canal A en 2 y 3, canal B en 4 y 5).

### 2.3 Software

- `lector_encoders.py` : muestra los ángulos en vivo, calcula la posición de la punta con el modelo y registra poses para la validación.
- `serial_to_csv_encoder.py` : guarda un CSV con una fila por articulación y por muestra.
- `brazo_cinematica.py` : modelo del brazo (necesario para los dos scripts).
- Requisitos: Arduino IDE, Python 3, `pip install numpy pyserial` .

## 3. Cómo funciona

1. **El encoder va dentro del motor** y entrega dos ondas cuadradas (canales A y B) desfasadas 90°. Esas señales llegan al driver, que las usa para su control de lazo cerrado.
2. **El Arduino monitor se conecta en paralelo** a los terminales `EA+` , `EB+` y `EGND` del driver. Solo escucha; no envía nada.
3. **Cada cambio del canal A dispara una interrupción.** Si A y B tienen el mismo nivel, la cuenta sube en uno; si no, baja en uno. Así se obtiene también la dirección del giro.
4. **Cada 20 ms** el Arduino envía por USB las cuentas de todos los canales.
5. **Python convierte las cuentas en grados de articulación:**
   - ángulo = signo × cuentas ÷ (2 × PPR × reducción) × 360.
   - `PPR` , `REDUCCION` y `SIGNO` están en una tabla al inicio de `lector_encoders.py` .
6. **Con los ángulos, el modelo calcula la posición de la punta.** La articulación 6 no cambia esa posición, solo la orientación.
7. **El cero es relativo.** Las cuentas arrancan en 0 cuando la placa se reinicia, y **abrir el puerto serial desde Python reinicia el Arduino**. Por eso la pose del brazo en el momento de abrir el puerto (o de enviar `Z` ) es el ángulo cero. Hay que hacerlo con el brazo en la pose home.

## 4. Procedimiento

### Antes de empezar

- Contar con la aprobación del asesor: se agregan cables a los terminales de los drivers.
- Ubicar el paro de emergencia y trabajar en los terminales solo con el sistema **apagado**.
- Material: Arduino Mega nuevo con cable USB, multímetro, resistencias de 1 kΩ (una por señal), cables de puente y cinta aislante.

### Paso 1: Prueba sobre la mesa, sin tocar el brazo

Subir el sketch al Mega nuevo, abrir el Monitor Serial del IDE a 115200 y tocar con un cable el pin 2 contra GND y contra 5 V. Cerrar el Monitor Serial al terminar.

**Esperado:** unas 50 líneas por segundo del tipo `E,<ms>,0,0,0,0,0` ; la primera cuenta cambia en ±1 con cada cambio de nivel y las otras cuatro se quedan en 0.

### Paso 2: Medir el voltaje con el sistema encendido

Con el multímetro en DC, medir en un driver: `EA+` contra `EGND` , `EB+` contra `EGND` , `VCC` contra `EGND` y el GND del Arduino monitor contra `EGND` .

**Esperado:** con el motor quieto, cada señal marca cerca de 0 V o cerca de 5 V; al girar despacio cambia entre ambos niveles y, si gira rápido, el multímetro muestra un valor intermedio (cerca de 2.5 V). `VCC` cerca de 5 V y casi 0 V entre las tierras.

**Criterio para detenerse:** si cualquier señal supera 5 V en algún momento, no conectar nada.

#### Paso 3: Conexión, con el sistema apagado

Empezar con una sola articulación (la 4 o la 5, que tienen resolución y colores de cable confirmados). Conectar `EA+` y `EB+` con su resistencia de 1 kΩ a los pines de la tabla 2.2, y `EGND` al GND del Arduino. No conectar el `VCC` del encoder. Si se agrega un cable bajo un tornillo ya ocupado, evitar que toque los terminales vecinos y aislar las puntas.

#### Paso 4: Primera lectura

Con el brazo en home: `python lector_encoders.py --puerto COM4 --activas 4 --cero` (cambiar el puerto). Mover la articulación un poco con la pantalla y regresarla.

**Esperado:** el ángulo empieza en 0.00°, cambia de forma suave con el movimiento, y al regresar vuelve a cerca de 0 con una diferencia de pocas cuentas (centésimas de grado). Si el ángulo baja cuando debería subir, se cambia el signo en la tabla `SIGNO`.

#### Paso 5: Prueba de una vuelta del motor

Obligatoria para las articulaciones 1 a 3. Con el sistema apagado y un encoder conectado, correr el script con `--registro datos.csv`, girar el eje una vuelta a mano y mirar el cambio de cuentas.

**Esperado:** unas 2000 cuentas. Si sale otra cifra, se corrige `PPR` (unas 1000 cuentas indican 500 pulsos por vuelta; unas 4000, 2000).

#### Paso 6: Agregar las demás articulaciones

Una a la vez, repitiendo los pasos 2 a 4. Al final, ejecutar con `--activas 1,2,3,4,5`.

#### Paso 7: Guardar datos en CSV

```
python serial_to_csv_encoder.py --port COM5 --activas 4,5 --cero
```

Opciones útiles: `--solo-cambios` para guardar solo las filas donde la cuenta cambió, y `--simular` para probar sin hardware.

### 5. Qué se espera recibir

**Del Arduino monitor** (cada 20 ms): `E,123456,-1520,0,38,4210,-96`, es decir, el tiempo en milisegundos y las cuentas de los cinco canales. Los canales sin conectar deben quedarse en 0.

**Del CSV** (una fila por articulación y por muestra):

- `timestamp_us`, `driver_id` (número de articulación), `source` (siempre `E`), `direction` (−1, 0 o +1), `position` (cuenta), `dt_us`, `velocity_pps` (cuentas por segundo con signo), `angle_deg` y `velocity_dps` (grados por segundo).
- A la velocidad del modo manual cada articulación genera entre 1250 y 1570 cuentas por segundo, es decir, entre 25 y 31 cuentas por muestra.

**En pantalla:** los ángulos de cada articulación en vivo y la posición de la punta según el modelo.

## 6. Validación contra el modelo

### 6.1 Con regla

1. Con el brazo en una pose, marcar con lápiz sobre una hoja el punto donde queda la punta y presionar Enter en el script para registrar la pose.
2. Mover a otra pose, marcar el punto y presionar Enter otra vez.
3. Medir con regla la distancia entre marcas y compararla con la que imprime el modelo. Repetir con varias parejas.

**Esperado:** distancias parecidas. Diferencias de milímetros a un par de centímetros pueden deberse a la regla y a la holgura de los engranes (como orientación, no como criterio estricto). Diferencias grandes y repetidas indican un problema de fondo: las longitudes provisionales `13` y `14` del modelo, un signo invertido o una resolución mal supuesta.

### 6.2 Con una secuencia automática del Arduino (ángulos conocidos)

`Codigo_final.ino` trae secuencias que mueven ángulos exactos y sirven de referencia sin medir nada. **Esta prueba mueve el brazo de forma autónoma**, además del servo y la pinza.

Preparación: encoders de las articulaciones 1 y 3 conectados, brazo en home, área despejada, paro de emergencia al alcance y `python lector_encoders.py --puerto COM4 --activas 1,3 --cero --registro auto1.csv`. Desde la pantalla, modo automático y secuencia 1.

**Esperado en la secuencia 1** (convención del programa):

- Articulación 1: −30°, −60° y −90° en los tres pasos, y regresa a 0° al final.
- Articulación 3: en cada paso recorre 0°, +60°, +80°, +40°, +60° y vuelve a 0°.

La secuencia 2 permite verificar las articulaciones 2 y 5 ( `--activas 1,2,3,5` ): J1 −15°, J3 −40°, J5 +50° y J2 +5°, y regresa en orden inverso.

**Cómo interpretar:** el ángulo mostrado es el esperado multiplicado por (PPR real ÷ PPR supuesto). Si sale el doble o la mitad, se corrige `PPR` con PPR real = PPR supuesto × (lectura ÷ esperado). Si solo cambia el signo, se corrige `SIGNO`. Si la articulación 3 no regresa a 0° tras cada paso, se perdieron cuentas.

## 7. Ruido y problemas frecuentes

- **La cuenta oscila entre dos valores con el motor quieto (+1, −1, +1).** Normal: el encoder está justo en un borde. Equivale a milésimas de grado.
- **Saltos de cientos o miles de cuentas mientras el motor trabaja.** Ruido eléctrico de los drivers en los cables del encoder. Usar cables cortos, alejarlos de los cables de potencia, verificar la tierra común y, si es posible, usar cable trenzado o blindado.
- **La cuenta se desplaza poco a poco con el motor quieto.** Ruido o contacto flojo. Revisar conexiones y tierra.
- **Al regresar a la pose inicial el ángulo no vuelve a cero.** Pueden haberse perdido cuentas. Repetir más despacio. Pocas cuentas de diferencia es normal; cientos, no.
- **La cuenta solo sube o solo baja.** El canal B probablemente no está conectado o está en el pin

equivocado.

- **La cuenta no cambia aunque el motor gira.** Revisar el canal A, el pin elegido y el voltaje de la señal.
- **El ángulo cambia el doble o la mitad.** La resolución supuesta no es la real (paso 5).
- **El sentido está invertido.** No es ruido; se corrige con `SIGNO`.
- **Un canal sin conectar cambia.** No debería ocurrir con las resistencias internas; revisar que no haya un cable suelto tocando ese pin y excluirlo con `--activas`.
- **No llegan datos o aparece "Access is denied".** Otro programa (por ejemplo el Monitor Serial del IDE) tiene abierto el puerto.
- **Los ángulos "cambian solos" justo al abrir Python.** Al abrir el puerto el Arduino se reinicia y las cuentas vuelven a 0: es el comportamiento esperado.

## 8. Riesgos y cómo reducirlos

| Riesgo | Posible consecuencia | Prevención |
|---|---|---|
| Señal de encoder mayor a 5 V | Daño del pin o de la placa del Arduino | Medir antes (paso 2); no conectar si supera 5 V; usar la resistencia en serie |
| Cable que toca un terminal vecino o un corto | Alarma o daño del driver | Conectar con el sistema apagado, aislar puntas, revisar dos veces antes de encender |
| Carga o error en la línea de encoder | El driver pierde su referencia o entra en alarma (según el manual del CL57Y, la luz roja con 5 destellos indica error de posición y con 6, encoder incorrecto) | Resistencias en serie, conectar una articulación a la vez, observar la luz del driver; desconectar si hay alarma |
| Alimentación mezclada | Daño o comportamiento extraño | No conectar el `VCC` del encoder al Arduino; alimentar el monitor solo por USB |
| Diferencia de tierras entre la computadora y el driver | Lecturas ruidosas o corrientes por la tierra | Medir entre el GND del Arduino y `EGND`: debe ser casi 0 V antes de conectar |
| Terminales de potencia del motor | Descarga o cortocircuito (los drivers trabajan entre 24 y 50 V DC) | Tocar solo los terminales de encoder y siempre con el sistema apagado |
| Movimiento autónomo del brazo en las secuencias automáticas | Golpes o daño al brazo o al entorno | Probar solo con el brazo en home, área despejada y paro de emergencia al alcance |
| El programa del Arduino no tiene límites de giro ni | Una articulación puede llegar a un tope si se sigue moviendo | Mover en pequeños tramos y observar; cualquier protección |

| | | |
|---|---|---|
| lleva la posición; en modo manual el paro no se revisa | | depende del PLC, cuyo programa no se tiene |
| Resolución de los NEMA 23 supuesta | Ángulos con el doble o la mitad del valor real | Prueba de una vuelta (paso 5) y secuencia automática |
| Cuentas perdidas por ruido | Ángulos desviados que se acumulan | Comprobar el regreso a cero; repetir más despacio |
| El encoder mide el motor, no la articulación | Diferencias con la posición real por holgura de engranes | Tenerlo en cuenta al comparar con la regla |
| Cero perdido al reiniciar el Arduino | Ángulos desplazados | Reiniciar o enviar `Z` siempre con el brazo en home |
| Longitudes `l3` y `l4` provisionales del modelo | Posición de la punta incorrecta | Medirlas en el brazo y corregir `brazo_cinematica.py` |
| Falta de autorización | Problemas con el equipo responsable del brazo | Obtener la aprobación del asesor antes de conectar nada |

## 9. Supuestos pendientes de confirmar

- Que las señales `EA+` y `EB+` sean de 5 V (paso 2).
- Que los motores NEMA 23 tengan 1000 pulsos por vuelta (paso 5).
- Qué driver mueve cada articulación (deducido de la tesis; confirmar con el cableado).
- Que los drivers estén en 800 pulsos por vuelta (interruptores DIP).
- Que los ángulos del modelo coincidan con los de la tesis y su convención de signo (validación del paso 6).
- Las longitudes `l3` y `l4` del modelo.